



Estructura de Datos.
Examen Convocatoria Ordinaria
Duración: 2:00 horas

Nombre y apellidos:

Nota aclaratoria: está prohibido el uso de la librería STL en el presente examen, utilizar dicha librería supondría la NO puntuación del ejercicio.

1. **(1 punto)** Responde las siguientes preguntas relacionadas con la eficiencia de los algoritmos:
 - a. **(0,75 puntos)** Define las notaciones O , Ω y Θ , y explica cómo se relacionan con el análisis del caso mejor, el caso peor y el caso promedio de un algoritmo.
 - b. **(0,25 puntos)** Dame un ejemplo de cada caso (caso mejor, caso peor y caso promedio) suponiendo que queremos realizar un algoritmo de búsqueda con un vector.
2. **(1,5 puntos)** Responde a las siguientes preguntas relacionada con la eficiencia de algoritmos:
 - a. **(1 punto)** Dado el siguiente algoritmo de búsqueda:

```
int binarySearch(int arr[], int size, int target) {
    int left = 0, right = size - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid; // Elemento encontrado
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1; // Elemento no encontrado
}
```

¿Qué eficiencia tiene el método teniendo en cuenta el peor caso? ¿Por qué?
¿Cuál sería el mejor caso? Explica que situación se daría y qué eficiencia sería.
 - b. **(0,5 puntos)** Si el algoritmo anterior lo programo en Java, ¿Será más rápido que el proporcionado en C++? ¿Será más eficiente? Explica y **justifica suficientemente tu respuesta**.
3. **(1,5 puntos)** Resuelve las siguientes cuestiones:
 - a. **(1 punto)** Construye un árbol AVL realizando las siguientes operaciones, indicando todos los pasos y las rotaciones que se realizan en caso de que sean necesarias.
 - i. Inserta los valores: 40, 20, 60, 10, 30, 70, 50, 25, 15, 65.
 - ii. Elimina los valores: 60, 30, 70.
 - iii. Inserta los valores: 55, 35.
 - b. **(0,5 puntos)** Listar los nodos del árbol resultante en los recorridos Preorden, Inorden y Postorden.

4. **(2 puntos)** Dado un árbol binario de búsqueda con interfaz:

```
template <class T>
class ABB {
public:
    ABB();
    ABB(T valor);
    Arbol(const ABB<T> & a);
    ~ABB();
    bool insertar (T valor);
    bool borrar(T valor);
    void vaciar();
    int numNodos() const;
    bool estaVacio() const;
    ABB<T> & operator = (const ABB<T> &a);
};
```

- a) (0,75 punto) Implementa la función de búsqueda de manera recursiva. Utiliza las propiedades de los ABB para optimizar la búsqueda. Explica brevemente cómo funciona tu implementación.
 - b) (0,75 puntos) Implementa la función de búsqueda de manera iterativa. Explica brevemente cómo funciona tu implementación.
 - c) (0,5 puntos) Compara ambas implementaciones en términos de ventajas y desventajas.
5. **(2,5 puntos)** Un compilador necesita validar expresiones matemáticas para comprobar que los paréntesis, corchetes y llaves están correctamente balanceados. Para ello, se utilizará una pila como estructura principal.
- a. **(0,5 puntos)** Especifica la interfaz del **T.D.A. PilaBalanceo**, que incluye al menos las operaciones básicas de una pila (push, pop, top) y un método adicional para comprobar si una expresión está balanceada.
 - b. **(0,5 puntos)** ¿Cuál es su función de abstracción y el invariante de representación?
 - c. **(1 punto)** Implementa la función esBalanceada que recibe una expresión matemática en forma de cadena (por ejemplo, "(a + b) * [c - d]") y devuelve un valor booleano indicando si los delimitadores (paréntesis, corchetes y llaves) están correctamente balanceados.
 - d. **(0,5 puntos)** Analiza la eficiencia teórica de la función esBalanceada. ¿Qué ocurriría en el caso extremo de una expresión que contenga únicamente delimitadores de apertura (por ejemplo, (((([{{{}})?
6. **(1,5 puntos)** Define la clase Conjunto como una especialización del T.D.A. Lista mediante herencia. La clase debe sobrescribir el método de inserción para evitar duplicados, utilizando el método pertenece definido en la clase base. Realiza los siguientes apartados teniendo en cuenta que debes incluir la interfaz de **ambas clases (Lista y Conjunto)** y todo lo necesario para que el programa funcione.
- a. (1 punto) Define la clase Conjunto como una especialización del T.D.A. Lista mediante herencia. La clase debe sobrescribir el método de inserción para evitar duplicados, utilizando el método pertenece definido en la clase base.
 - b. (0,5 puntos) Implementa el método insertar, asegurándote que no existan elementos duplicados. ¿Cómo incluirías esta especificación en la clase?